

Aadhaar App Intent Integration Specification

Version	Version Date	Created by	Reviewed by	Status
V1	17-Feb	Dhinavahi Vasant Kalyan	Abee Narayan	Updated
V2	16-April	Dhinavahi Vasant Kalyan	Abee Narayan	Deployed

Aadhaar App Intent Integration Specification	1
1. Introduction	3
2. Before You Start	3
2.1 Cryptographic Setup.....	3
2.2 OVSE Registration	3
2.3 Backend Requirements.....	4
3. Steps for intent generation	5
3.1 Create the Intent Request JWT.....	5
3.1.1 JWT Header	5
3.1.2 JWT Payload Structure	5
3.1.3 Payload Field Specification	6
3.1.4 Complete Bitmap Mapping.....	8
3.1.5 Language Mapping Table	9
3.2 Choose Invocation Method	10
3.2.1 App to App	10
3.2.2 Web to App	11
3.2.3 Cross Device QR	11
3.3 Receive SD-JWT.....	11
3.4 Verify SD-JWT	12
3.4.1 Signature Verification	12

3.4.2 Disclosure Verification.....	12
3.5 Send Acknowledgement	12
3.6 Security Requirements.....	14
3.7 Final Integration Checklist.....	14

1. Introduction

This document provides a complete end-to-end guide for integrating with Aadhaar App for Verifiable Credential exchange.

The integration supports three invocation mechanisms:

1. App to App flow
2. Web to App flow
3. Cross Device QR flow

The JWT request structure remains common across all flows. Only the invocation method differs.

2. Before You Start

Before beginning integration, ensure the following prerequisites are completed.

2.1 Cryptographic Setup

You must generate an RSA key pair.

- 1) Private key → used to sign Intent Request JWT
- 2) Public key → shared with UIDAI during onboarding

Requirements:

- 1) Algorithm: RSA
- 2) Signing: RS256
- 3) Private key must remain in backend
- 4) Private key must never be embedded in frontend or mobile apps

2.2 OVSE Registration

To obtain OVSE Client ID, you must provide UIDAI:

- 1) RSA Public Key
- 2) Logo in SVG format (Less than 10 KB)
- 3) Callback URL
- 4) Domain Name
- 5) Entity Name to be displayed inside Aadhaar App

UIDAI will register your public key and issue OVSE Client ID. Integration cannot proceed without registration.

2.3 Backend Requirements

Your backend must support:

- 1) SD-JWT creation and parsing
- 2) RS256 signature verification
- 3) SHA256 hashing
- 4) Base64URL encoding
- 5) HTTPS callback endpoint
- 6) Secure private key storage

2.4 JWT Signature Validation (Recommended Pre-check)

Before sharing the public key and proceeding with integration, OVSEs are strongly recommended to validate the JWT signature format and structure.

Steps:

1. Generate a sample JWT using your private key
2. Verify the JWT on <https://jwt.io>
3. Ensure:
 - a. Algorithm is RS256
 - b. Signature is correctly verifiable using the public key
 - c. Payload structure matches UIDAI specification

This step helps identify implementation issues early and reduces integration failures.

(This is for development validation only and should not be used in production flows.)

3. Steps for intent generation

3.1 Create the Intent Request JWT

Every integration begins by constructing a signed JWT.

3.1.1 JWT Header

```
{  
  "alg": "RS256",  
  "typ": "JWT"  
}
```

Algorithm must be RS256.

3.1.2 JWT Payload Structure

```
{  
  "txn": "94f63917-708a-4e58-9001-fce56d839948",  
  "i": "credential",  
  "lang": "23",  
  "sc": "1110001110001110000001110001110001110001111",  
  "pop": 1,  
  "ch": "app",  
  "m": "1",  
  "ac": "1a2f",  
  "sa": "00b1",  
  "cb": "https://verifier.com/api/callback",  
  "aud": "https://verifier.com",  
  "iss": "https://uidai.gov.in",  
  "exp": 1747094073,  
  "iat": 1747093773,  
  "ht": "RAM KUMAR",  
}
```

```

    "aid": "in.gov.uidai.samvaad",
    "asig": "BASE64_SIGNATURE",
    "jti": "8a9d7c2b-b932-4131-9e36-43f22a019be5"
  }

```

3.1.3 Payload Field Specification

Below is the authoritative field specification.

Field	Type	Format	Allowed Values	Mandatory	Applies To	What You Must Do
txn	String	UUID	Any valid UUID	Yes	All flows	Generate new UUID per request. Never reuse.
i	String	Literal string	"credential"	Yes	All flows	Must always be "credential".
lang	String	Numeric string	1–23	Yes	All flows	Pass numeric language code. Defaults to 23 if omitted.
sc	String	64-bit binary string	0 or 1 per bit	Yes	All flows	Construct full 64-bit bitmap. Set only required claims to 1.
pop	Integer	0 or 1	0 or 1	Yes	All flows	Set to 1. Face authentication required.
ch	String	Literal string	"app", "web", "qr"	Yes	All flows	Must match invocation method.

m	String	"0" or "1"	"0", "1"	Yes	All flows	Set to "1" for online authentication.
ac	String	Alphanumeric	OVSE Client ID	Yes	All flows	Use value issued by UIDAI if applicable.
sa	String	Alphanumeric	OVSE Registration number	Yes	All flows	Use code issued during onboarding.
cb	String	HTTPS URL	Valid URL	Yes	Web to App, Cross Device	Must be HTTPS and publicly accessible.
aud	String	URL (domain only)	Must match cb domain	Yes	Web to App, Cross Device	Must equal callback domain.
iss	String	URL	Fixed value	Yes	All flows	Use configured issuer value.
exp	Integer	Unix timestamp	Future epoch time	Yes	All flows	Set to short expiry, recommended 5 minutes.
iat	Integer	Unix timestamp	Current epoch time	Yes	All flows	Set to current timestamp.
ht	String	Plain text	Any string	Yes but can be a null value	All flows	Use as profile hint in multi-profile scenarios.
aid	String	Package ID	App ID	Yes Yes but can be a null value	App to App	Mandatory for App flow. Provide application ID.

asig	String	Base64 string	Encoded signature	Yes but can be a null value	App to App	Mandatory for App flow. Provide Base64 app signature.
jti	String	UUID	Any valid UUID	Yes	All flows	Generate unique JWT ID per request. Used for replay protection.

3.1.4 Complete Bitmap Mapping

Bit	Claim
1	credentialIssuingDate
2	enrolmentDate
3	enrolmentNumber
4	isNRI
5	residentImage
6	residentName
7	localResidentName
8	ageAbove18
9	ageAbove50
10	ageAbove60
11	ageAbove75
12	dob
13	gender
14	careOf
15	localCareOf
16	building
17	localBuilding
18	locality
19	localLocality
20	street

21	localStreet
22	landmark
23	localLandmark
24	vtc
25	localVtc
26	subDistrict
27	localSubDistrict
28	district
29	localDistrict
30	state
31	localState
32	poName
33	localPoName
34	pincode
35	address
36	regionalAddress
37	mobile
38	maskedMobile
39	email
40	maskedEmail
41-63	Reserved padding

You must always construct a full 64 bit string.

3.1.5 Language Mapping Table

Code	Language
1	Assamese
2	Bengali
3	Bodo
4	Dogri
5	Gujarati

6	Hindi
7	Kannada
8	Kashmiri
9	Konkani
10	Maithili
11	Malayalam
12	Manipuri
13	Marathi
14	Nepali
15	Oriya
16	Punjabi
17	Sanskrit
18	Santhali
19	Sindhi
20	Tamil
21	Telugu
22	Urdu
23	English

3.2 Choose Invocation Method

3.2.1 App to App

Android:

```
val intent = Intent("in.gov.uidai.pehchaan.INTENT_REQUEST").apply {
    putExtra("request", jwt)
}
startActivity(intent)
```

iOS:

```
UIApplication.shared.open(URL(string:
"pehchan://in.gov.uidai.pehchan?req=\\(jwt)")!)
```

3.2.2 Web to App

```
window.location.replace(
"intent:#Intent;action=in.gov.uidai.pehchaan.WEB_INTENT_REQUEST;S.request=" + jwt + ";end"
);
```

3.2.3 Cross Device QR

Steps:

1. Sign JWT
2. Encode ISO-8859-1
3. Append delimiter 255
4. Gzip compress
5. Convert to BigInt
6. Convert to Base10
7. Prefix: <https://maadhaar.com/getIntent?value=<BASE10>>
8. Generate QR

3.3 Receive SD-JWT

Callback format:

```
<Request>
  <TxnID>TXN_ID</TxnID>
  <Credential>SD-JWT</Credential>
</Request>
```

Format:

a.b.c~d1~d2

3.4 Verify SD-JWT

3.4.1 Signature Verification

1. Split header.payload.signature
2. Retrieve UIDAI public key
3. Verify RS256 signature
4. Reject if invalid

3.4.2 Disclosure Verification

For each disclosure:

1. Hash disclosure string using SHA256
2. Base64URL encode
3. Match against _sd array
4. Reject if mismatch

Both validations are mandatory.

3.5 Send Acknowledgement

```
<Response>  
  <TxnID>TXN_ID</TxnID>  
  <ResponseCode>200</ResponseCode>  
  <ResponseMsg>Success</ResponseMsg>  
</Response>
```

For failure scenarios:

```
<Response>
  <TxnID>TXN_ID</TxnID>
  <ResponseCode><ERROR_CODE></ResponseCode>
  <ResponseMsg><ERROR_MESSAGE></ResponseMsg>
</Response>
```

3.6 Error Codes and Failure Scenarios

The following error codes standardize failure handling across Aadhaar App authentication flows. OVSE systems must handle these appropriately and provide meaningful user messaging.

Name	Case	Error Code	Error Message
User Reject	User explicitly taps Deny / Reject	301	User rejected the authentication request
Face Liveness Fail	Face liveness detection fails repeatedly, exceeding retry limit	302	Face liveness verification failed after 3 attempts
User Abort	User exits flow before completion (back press, app switch, close)	303	Authentication process aborted by user
Face Mismatch	Face captured successfully but does not match Aadhaar records	304	Face authentication failed due to mismatch with registered data
Biometric Lock	Biometric lock enabled in UIDAI system	305	Biometric authentication is locked for this user
FaceRD Server Down	FaceRD service unavailable	306	FaceRD service is currently unavailable. Please try again later
HTTP Error	Network/API failure (timeout, 5xx, invalid response)	307	Failed to complete authentication due to network or server error

Callback Retry	Auth success but callback delivery failed	308	Authentication completed but callback delivery failed. Retrying
-------------------	--	-----	---

3.7 Security Requirements

1. Private key must remain backend
2. Callback must use HTTPS
3. exp must be short lived
4. No credential logging
5. Validate txn and jti
6. Reject replayed transactions

3.8 Final Integration Checklist

1. RSA key pair generated
2. OVSE registered
3. JWT payload correctly constructed
4. Bitmaps correct
5. Language code corrects
6. JWT signed with RS256
7. Correct invocation method
8. SD-JWT verified fully
9. Acknowledgement sent